Goal of the project

In this project, a GPU Huffman decoder is implemented and compared to a CPU decoder. The GPU uses a gap array[1] that is implemented and tested. The encoded data is generated using [1] that includes a CPU decoder that is used as a comparison.

What is Huffman decoding?

Huffman encoding is a lossless source encoding algorithm. It works by assigning a variable length code to a value depending on the amount of occurrences in the data. Values that are very frequent in the data will have a shorter length code assigned than infrequent values. When all the values have a designated code, the codes are sometimes represented as a tree, as the codes used are prefix codes [2]. Prefix codes differ from one another as the prefix from one code cannot be a code in itself, doing this no confusion exists when decoding. Using the assigned code's the encoded code can be decoded to construct back the original bit sequence.

Project implementation

When using a GPU to decode Huffman code, the main difficulty is knowing where the start of a character is, since the characters have a varying length. One solution to this is to divide the encoded array in even sections, where each section is decoded on a thread. As the threads don't necessarily start at the beginning of a character, many of the decoded value's may not be correct. The decoded characters can however after decoding some bits be correct. In this situation, we need a certain amount of overlap between the sections to verify if the decoded section is correct. This overlap significantly reduces the gained efficiency of using multiple threads. Alternatively, a gap array can be used. The gap array, like the one used in this project, is an additional array send with the Huffman code that includes the bit size of each character in the encoded sequence. Using this gap array, a thread knows with certainty where to start decoding correctly, thus not requiring an overlap. Figure 1 shows the encoded bit sequence with gap array, where the gap value indicates the length of each code.

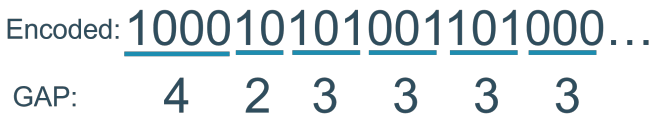


Figure 1: Data visualization

The encoder from 1 was altered to create a gap array that can be used by the GPU decoder. Using this gap array, additional data is being sent, but the decoding speed can be increased according to [1]. In this code, the encoded array is splits into blocks of a variable size. Each thread decodes a block, where it begins decoding at the start of a character. The start of a character inside the block is retrieved from the gap array. After the beginning of a character is found, the GPU starts to decode the bits. To decode the data, the GPU compares a bit sequence to that of a code. When a match is found between the bit sequence and a code, the GPU writes the corresponding character to an output array.

Performance of the GPU decoder

To test the performance of the decoder 2 example texts are decoded with a word count of 5000 and 10000. The results of the decoding are presented in Table 1 and Table 2. To measure the total decoding time the opening and reading of the files is not included in the time, only the time the decoder uses is measured. The measured time is the average of 100 iterations.

[1] https://github.com/adeesha-savinda/huffman-encode-decode
[2] https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/

Device	Time (s)	block size
CPU	0.008241	/
GPU	0.101500	8
GPU	0.033821	32
GPU	0.029195	128
GPU	0.027750	256
GPU	0.043949	512

Table 1: 5000 words text

Device	Time (s)	block size
CPU	0.004057	/
GPU	0.040590	8
GPU	0.019107	32
GPU	0.013889	128
GPU	0.021855	256
GPU	0.038763	512

Table 2: 10000 words text

Conclusion

The current implementation is around 4-5 times slower at best compared to the CPU decoder. The block size used to decode significantly impacts the overall decoding time of the GPU decoder. Block sizes of 128 or 256 seem to perform best in these test. The better performance of bigger blocks is likely due to the decrease in overhead for thread creation and execution.

Possible improvements

Currently, the decoder is not fully optimized and as a result slower compared to the CPU decoder. By optimizing the decoder, we could expect results that are closer to [1]. The gap array size could be reduced by creating a gap array based on the beginning of the first character within a block instead of the beginning of each character. This is needed as the current solution uses a lot of storage space, which is not desired for an encoder.

References

[1] Naoya Yamamoto et al. "Huffman Coding with Gap Arrays for GPU Acceleration". en. In: *49th International Conference on Parallel Processing - ICPP*. Edmonton AB Canada: ACM, Aug. 2020, pp. 1–11. ISBN: 978-1-4503-8816-0. DOI: 10.1145/3404397.3404429. URL: https://dl.acm.org/doi/10.1145/3404397.3404429 (visited on 05/09/2024).